

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Annexure A

Total Marks:20

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I X. Joshua (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

X. Joshua

Annexure A

Short Answer Test

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.

Answer:

I would design a Pandas and NumPy preprocessing pipeline with the following steps:

1. **Load the data** using `pd.read_csv()` and inspect it using `head()`, `info()`, and `isnull().sum()`.
2. **Remove duplicate records** using `drop_duplicates()`.
3. **Handle missing values** using median for numerical columns and mode or "Unknown" for categorical columns.
4. **Standardize units**, such as converting temperature to °C and weight to kg.
5. **Filter invalid values** using Boolean indexing. For example, patient age should not be negative.
6. **Standardize categorical values** using `str.strip()`, `str.lower()`, and `replace()`.
7. **Use indexing** to select required rows and columns efficiently.
8. **Use aggregation** such as `mean()`, `median()`, `min()`, and `max()` to generate reports.
9. Check for unexpected anomalies using range checks and statistical methods such as Z-score or IQR.
10. **Save the cleaned data** using `to_csv()`.

For near real-time processing, vectorized Pandas and NumPy operations should be used instead of loops. Data can be processed in batches when the files are very large. A validation step should be performed before saving to ensure that no critical errors remain.

2. A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Answer:

I would use Pandas for structured transaction data and NumPy for efficient numerical operations.

1. Load only the required columns using `usecols`.
2. Use appropriate data types such as `float32` where suitable to reduce memory usage.
3. Convert transaction amounts and timestamps to proper types.
4. Filter invalid transactions using Boolean indexing.
5. Group transactions by customer or account using `groupby()`.
6. Calculate `sum()`, `mean()`, `std()`, `min()`, and `max()` to identify unusual spending.
7. Use NumPy vectorized operations for numerical calculations.
8. Set frequently searched columns as indexes when appropriate.
9. Avoid unnecessary copies and intermediate DataFrames.

10. Process very large files in chunks using chunksize.

For example:

```
summary = df.groupby("customer_id")["amount"].agg(
    ["sum", "mean", "std", "count"]
)
```

Performance can be debugged by measuring execution time, checking memory consumption, and identifying slow operations. Incorrect results can be checked by validating data types, missing values, duplicate transactions, and comparing calculations on a small sample.

3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?

Answer:

First, I would read each file according to its format, such as CSV, Excel, or JSON.

```
sales = pd.read_csv("sales.csv")
```

```
marketing = pd.read_excel("marketing.xlsx")
```

```
support = pd.read_json("support.json")
```

The column names and data types should be standardized before merging. A common key such as customer_id should be identified and checked for duplicates.

If all records need to be preserved, an outer merge can be used:

```
merged = sales.merge(
    marketing,
    on="customer_id",
    how="outer"
)
```

The support data can then be merged in the same way. Duplicate keys can be detected using duplicated() and groupby(). Missing keys can be identified using isna() and filtering.

Finally:

```
merged = merged.sort_values("customer_id")
```

```
merged = merged.reset_index(drop=True)
```

Conflicts can be resolved using source-priority rules, valid-value selection, or aggregation. Record counts, missing values, duplicate IDs, and important columns should be validated before saving the final DataFrame.

4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.

Answer:

The workflow would contain the following stages:

1. File Loading:

Read daily files using Pandas functions such as read_csv(), read_excel(), or read_json().

2. Error Handling:

Use exception handling to detect corrupted files and check whether required columns exist.

3. Data Cleaning:

Convert data types, handle missing values, remove duplicates, and standardize formats.

4. Filtering:

Use Boolean indexing to select relevant records.

5. Aggregation:

Use groupby() with functions such as sum(), mean(), and count().

```
summary = df.groupby("category")["sales"].agg(  
    ["sum", "mean", "count"]  
)
```

6. Sorting:

Sort the summary using sort_values().

7. Visualization:

Use Matplotlib to create bar charts, line charts, or histograms.

8. Output:

Save the processed dataset and summary report using to_csv() or to_excel().

Corrupted files should be logged and skipped or flagged. Missing columns should generate an error message. Inconsistent formats should be standardized before analysis.

5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

Answer:

I would first load the clickstream data into a Pandas DataFrame and inspect its structure.

1. Convert timestamps using pd.to_datetime().
2. Remove duplicate click events.
3. Handle missing values using suitable techniques.
4. Standardize text and categorical values.
5. Filter invalid events using Boolean conditions.
6. Set timestamp or user ID as an index when useful.
7. Extract features such as hour, day, session duration, number of clicks, and number of products viewed.
8. Group data by customer or session to create behavioral features.
9. Encode categorical variables.
10. Normalize numerical features before machine learning.

For example:

```
df["timestamp"] = pd.to_datetime(df["timestamp"])  
df["hour"] = df["timestamp"].dt.hour  
df = df.drop_duplicates()
```

The final DataFrame should contain clean, consistent, numerical features suitable for a machine-learning model.

6. A healthcare analytics system requires continuous monitoring of patient vitals stored as time-series data. The dataset includes irregular timestamps and noisy readings. Describe how you would use NumPy and Pandas to clean, resample, and analyze this

data. Explain how rolling statistics and aggregation functions can help identify abnormal patterns.

Answer:

First, I would convert timestamps into Pandas datetime objects and set them as the index.

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
```

```
df = df.set_index("timestamp").sort_index()
```

Invalid readings can be removed using filtering based on medically valid ranges. Irregular readings can then be resampled at fixed intervals.

```
df = df.resample("1min").mean()
```

Short missing intervals can be handled using interpolation.

Rolling statistics can detect changes in patient vitals:

```
df["rolling_mean"] = df["heart_rate"].rolling(5).mean()
```

```
df["rolling_std"] = df["heart_rate"].rolling(5).std()
```

Aggregation functions such as `mean()`, `max()`, `min()`, and `std()` can summarize patient measurements. Readings significantly different from the rolling mean can be flagged as possible abnormal patterns.

7. You are given a dataset with millions of rows containing mixed data types and categorical variables. The goal is to prepare this data for training a machine learning model under strict time constraints. Explain how you would perform encoding, normalization, and efficient data transformation using Pandas and NumPy. Discuss optimization and debugging strategies.

Answer:

I would first inspect the dataset using `info()` and identify numerical and categorical columns.

1. Convert numerical columns using `pd.to_numeric()`.
2. Handle missing numerical values using median.
3. Handle missing categorical values using mode or "Unknown".
4. Encode categorical variables using `pd.get_dummies()` or suitable encoding techniques.
5. Convert numerical data to efficient NumPy arrays.
6. Normalize numerical features using Min-Max or Z-score normalization.
7. Use vectorized operations instead of loops.
8. Process large files in chunks when necessary.
9. Remove unnecessary columns and duplicate records.
10. Validate the final feature matrix.

For example:

```
df = pd.get_dummies(df, columns=["gender", "city"])
```

```
X = df[numerical_columns].to_numpy(dtype=np.float32)
```

Performance can be improved by reducing memory usage and avoiding unnecessary copies. Debugging should check data types, missing values, array shape, encoded columns, and scaling results.

8. A logistics company wants to analyze delivery performance using large datasets containing location, time, and traffic data. Explain how you would design a data processing pipeline using Pandas for grouping, filtering, and aggregation. Discuss how indexing and sorting improve query efficiency and how you would validate the accuracy of computed metrics.

Answer:

The data should first be loaded into Pandas and cleaned.

1. Convert delivery timestamps to datetime.
2. Remove duplicate deliveries.
3. Handle missing location and traffic values.
4. Filter invalid delivery records.
5. Set frequently queried fields such as delivery time or delivery ID as indexes when appropriate.
6. Group records by location, region, driver, or date.
7. Calculate metrics such as average delivery time and number of deliveries.
8. Sort results based on delivery performance.

Example:

```
summary = df.groupby("region").agg(  
    average_time=("delivery_time", "mean"),  
    total_deliveries=("delivery_id", "count")  
)
```

```
summary = summary.sort_values(  
    "average_time"  
)
```

Accuracy can be validated by comparing aggregated totals with the original valid record count and checking minimum, maximum, and average values. Sample records should also be manually verified.

9. You are tasked with building a data validation system for incoming datasets in a machine learning pipeline. The system must check for missing values, invalid ranges, and inconsistent formats. Describe how you would implement validation rules using Pandas, and explain how indexing, filtering, and aggregation help enforce data quality constraints.

Answer:

I would create predefined validation rules for each column.

1. Check whether all required columns exist.
2. Verify expected data types using dtypes.
3. Check missing values using `isna().sum()`.
4. Check duplicate records.
5. Use filtering to detect values outside valid ranges.
6. Check categorical values against an allowed list.
7. Use aggregation to calculate minimum, maximum, mean, and count.
8. Use indexing to locate invalid records.
9. Store all errors in a validation report.
10. Allow the dataset to proceed only when critical constraints are satisfied.

For example:

```
invalid = df[  
    (df["age"] < 0) |  
    (df["age"] > 120)  
]
```

Aggregation can summarize the number of invalid records, while indexing allows the exact problematic rows to be identified.

10. A company wants to analyze user activity logs to generate insights such as peak usage times and user segmentation. The dataset is large and continuously updated. Explain how you would use NumPy and Pandas to process, filter, and aggregate this data efficiently. Discuss strategies for handling scalability and ensuring real-time performance.

Answer:

I would load the activity logs into Pandas and convert timestamps into datetime format.

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
```

```
df["hour"] = df["timestamp"].dt.hour
```

Invalid and duplicate logs would be removed using filtering and `drop_duplicates()`.

Peak usage can be calculated using grouping:

```
peak = df.groupby("hour")["user_id"].count()
```

Users can be segmented based on metrics such as number of sessions, clicks, or purchases.

NumPy can perform numerical calculations efficiently using vectorized operations. For scalability, only required columns should be loaded, appropriate data types should be used, and large files can be processed in chunks.

For real-time performance, incremental processing and efficient indexing should be used. Performance should be monitored by checking processing time and memory usage.

11. You are given a small dataset containing student records with missing marks and inconsistent grade formats. How would you use NumPy and Pandas to clean the data, fill missing values, and standardize grades? Explain your approach in simple steps.

Answer:

I would perform the following steps:

1. Load the dataset using Pandas.
2. Check missing values using `isna().sum()`.
3. Convert marks to numerical values using `pd.to_numeric()`.
4. Fill missing marks using the median or mean.
5. Standardize grades using `str.strip()` and `str.upper()`.
6. Replace inconsistent values such as "a", "A " and "grade a" with "A".
7. Remove duplicate student records.
8. Validate that marks are within the valid range.

Example:

```
df["marks"] = pd.to_numeric(
    df["marks"], errors="coerce"
)
```

```
df["marks"] = df["marks"].fillna(
    df["marks"].median()
)
```

```
df["grade"] = df["grade"].str.strip().str.upper()
```

Finally, the cleaned DataFrame can be saved for further analysis.

12. A company provides you with sales data, and you are asked to find the total and average sales for each product category. How would you use Pandas grouping and aggregation functions to solve this problem? Explain the steps clearly.

Answer:

First, load the sales data into a DataFrame and verify that the category and sales columns are available.

```
import pandas as pd
```

```
df = pd.read_csv("sales.csv")
```

Then group the data by product category and calculate the total and average sales.

```
summary = df.groupby("category")["sales"].agg(  
    total_sales="sum",  
    average_sales="mean"  
)
```

The result contains one row for each category with its total and average sales.

The output can be sorted:

```
summary = summary.sort_values(  
    "total_sales",  
    ascending=False  
)
```

Thus, `groupby()` divides the records into categories and `sum()` and `mean()` perform the required aggregations.

13. You receive a dataset with duplicate customer entries and incorrect data types (e.g., numbers stored as strings). How would you identify and fix these issues using Pandas? Describe the methods you would use for cleaning and validation.

Answer:

First, inspect the data types and duplicate records.

```
df.info()
```

```
print(df.duplicated().sum())
```

Duplicate records can be removed using:

```
df = df.drop_duplicates()
```

Numbers stored as strings can be converted using `pd.to_numeric()`.

```
df["age"] = pd.to_numeric(  
    df["age"],  
    errors="coerce"
```

```
)
```

After conversion, invalid values become missing values and can be handled appropriately.

I would then validate the cleaned dataset by checking:

```
df.isna().sum()
```

```
df.dtypes
```

```
df.duplicated().sum()
```

Finally, I would check numerical ranges and verify that customer IDs are unique where required.

14. A dataset contains timestamps and corresponding values, but the timestamps are not sorted. How would you use Pandas to sort the data and compute a simple moving average? Explain your approach.

Answer:

First, convert the timestamp column into datetime format.

```
df["timestamp"] = pd.to_datetime(  

```



```

    df["timestamp"]
)
Next, sort the data chronologically.
df = df.sort_values("timestamp")
A simple moving average can be calculated using the rolling() function.
df["moving_average"] = (
    df["value"].rolling(window=3).mean()
)

```

Here, window=3 means that the average is calculated using three consecutive observations. The first two rows will normally contain missing values because there are not enough observations to form a complete three-value window. This method provides an efficient way to calculate moving averages without manually implementing loops.

15. You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using NumPy and Pandas? Explain the steps briefly.

Answer:

The preprocessing can be performed in the following steps:

1. Load the dataset into a Pandas DataFrame.
2. Identify numerical and categorical columns.
3. Check missing values.
4. Fill missing numerical values using the median.
5. Fill missing categorical values using the mode.
6. Encode categorical variables.
7. Normalize numerical variables.
8. Convert the final numerical data into a NumPy array if required.
9. Validate the final dataset.

Example:

```

# Missing numerical values
df["age"] = df["age"].fillna(
    df["age"].median()
)

```

```

# Missing categorical values
df["city"] = df["city"].fillna(
    df["city"].mode()[0]
)

```

```

# Encoding
df = pd.get_dummies(
    df,
    columns=["city", "gender"]
)

```

For normalization, Min-Max scaling can be applied:

```

X = df[numerical_columns].to_numpy()

```

```

X_scaled = (
    X - X.min(axis=0)
)

```

```
) / (  
    X.max(axis=0) - X.min(axis=0)  
)
```

The resulting data contains numerical, consistent, and appropriately scaled features that can be supplied to a machine-learning model.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand